UCAM

Práctica P2-1
Introducción a Docker

Administración de Sistemas

Juan Morales García

Grado en Ingeniería Informática



Índice de contenidos

1. Introducción 2
2. Pasos Guiados 2
Contenedores 2
Imágenes 3
Dockerfiles 4
3. Ejercicios 4

1. Introducción

En este boletín veremos una primera aproximación a la virtualización por contenedores con Docker.

¡Recuerda! El password de la MV UbuntuDocker es **Asi4root**

2. Pasos Guiados

1. Ejecutamos `docker version` para ver la versión cliente y servidor de docker
2. Vemos detalles de la configuración de docker con

```
docker info
```

3. Vemos ahora los comandos que tenemos disponibles en docker simplemente ejecutando el comando `docker`

Contenedores

4. Vamos ahora a arrancar nuestro primer contenedor con el servidor web *nginx*

```
docker container run --publish 80:80 --detach nginx
```

5. Veamos los contenedores ejecutándose con

```
docker container ls
```

6. Paramos ahora el contendor con

```
docker container stop <id_del_contenedor>
```

7. Si ahora volvemos a ejecutar el sub-comando `ls` veremos que ya no nos aparece nada.
8. Pero si ejecutamos con la opción –a veremos todos los contenedores creados hasta ahora.

```
docker container ls -a
```

9. Volvemos a lanzar nuestro *nginx* pero esta vez le vamos a asignar un nombre

```
docker container run --publish 80:80 --detach --name webhost nginx
```

10. ¿Cuántos contenedores veremos si hacemos ahora un `docker container ls -a`?
11. Como tenemos nuestro servidor corriendo en segundo plano, podemos acceder a su log ejecutando

```
docker container logs webhost
```

12. Podemos también ver los procesos ejecutándose dentro de los contendores con

```
docker container top webhost
```

13. También podemos monitorizar nuestros recursos con

```
docker container stats
```

14. Borremos ahora todos los contenedores creados con

```
docker container rm <ids de los contenedores separados por espacio en blanco>
```

15. Vemos que no nos deja borrar uno pues está ejecutándose (deberíamos pararlo primero). También podemos borrar un contenedor funcionando con `rm -f`.
16. Ahora si hacemos `docker container ls -a` no nos debe aparecer ningún contenedor.
17. Vamos ahora a introducirnos en un contenedor para ejecutar algunos parámetros centro del mismo

```
docker container run -it --name proxy nginx bash
```

18. Entramos en un *bash* dentro de nuestro servidor web. Para poder salir simplemente deberíamos ejecutar `exit` (lo cual detiene el contenedor)
19. Probemos ahora crear un contenedor de ubuntu y acceder a él.

```
docker container run -it --name ubuntu ubuntu
```

20. Ahora podemos instalar cualquier paquete dentro de nuestro Ubuntu

```
apt-get update
```

```
apt-get install curl -y
```

21. Salimos del contenedor con `exit`.
22. Puedo entrar otra vez al contenedor con

```
docker container start -ai ubuntu
```

Imágenes

23. Veamos las imágenes que tenemos en nuestro sistema con

```
docker image ls
```

24. Vamos ahora a descargarnos una imagen diferente de *nginx* con

```
docker image pull nginx:1.11.9
```

25. Comprobemos las capas de *nginx*

```
docker image history nginx:latest
```

```
docker image inspect nginx:latest
```

26. Descarguémonos ahora una imagen que no sea oficial

```
docker image pull mysql/mysql-server
```

27. Si hacemos ahora un `docker image ls` vemos que el repositorio incluye *usuario/repositorio*

Dockerfiles

28. Vamos ahora al directorio */home/docker/EjemplosDocker* y entramos en el subdirectorio *dockerfile-sample-4*
29. Abrimos el fichero *Dockerfile* con Sublime Text.
30. Modificamos el valor de `CMD` de 10000 a 10
31. Ahora vamos a crear una imagen a partir de dicho fichero con

```
docker image build –t sleepylinux .
```

32. Si ahora hacemos `docker image ls` veremos que se nos ha creado una nueva imagen.
33. Vamos a ejecutar la imagen con `docker container run sleepylinux`
34. ¿Qué pasa si ejecutamos la imagen pasándole 5 como parámetro (`docker container run sleepylinux 5`)?

3. Ejercicios

Realiza de manera individual los siguientes pasos (*cada paso requiere la ejecución de un y sólo un comando*):

1. Descárgate la última imagen del servidor de correo `poste.io` del repositorio *docker hub*
2. Comprueba las capas de las que consta dicha imagen.
3. Arranca un contenedor basado en dicha imagen con nombre *correo-<nombre-del-alumno-y-apellidos>* (*ej: correo-fernando-terroso*). Debe ejecutarse en segundo plano.
4. Comprueba que el contenedor está ejecutándose.
5. Para el contenedor.
6. Comprueba que el contenedor no está ejecutándose.

7. Haz un listado de todos los contenedores creados independientemente de si se están ejecutando o no en este momento.
8. Ejecuta directamente un nuevo contenedor de `poste.io` pero esta vez con con la versión 2.2.24.
9. Escribe en un .txt cada uno de los comandos necesarios para cada paso y guárdalo en el escritorio con el nombre “tarea-Docker-1-nombre-apellidos.txt”. ***Importante que en la MV UbuntuDocker al hacer un listado de todos los contenedores aparezca los contenedores creados.***